Practical: 6	Write a program to implement Queue using arrays that perform the following operations. (a) Insert (b) Delete (c) Display (d) isEmpty (e) isFull
23/09/2025	

(a) Insert (b) Delete (c) Display (d) isEmpty (e) isFull

Program:

```
#include  <stdio.h>
#include <stdlib.h>
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;
int isEmpty() {
    return (front == -1);
}
int isFull() {
    return (rear == SIZE - 1);
}
void insert(int element) {
    if (isFull()) {
        printf("Queue is full! Cannot insert %d.\n", element);
        return;
    }
    if (isEmpty()) {
        front = 0;
    }
    queue[++rear] = element;
    printf("Inserted %d into the queue.\n", element);
}

void delete() {
    if (isEmpty()) {
        printf("Queue is empty! Cannot delete.\n");
        return;
    }
    printf("Deleted %d from the queue.\n", queue[front]);
    if (front == rear)
    {
         front = rear = -1;
    } else {
        front++;
    }
}
void display() {
    if (isEmpty()) {
        printf("Queue is empty!\n");
```

```
        return;
    }
    printf("Queue elements: ");
    for (int i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

int main() {
    int choice, element;

    while (1) {
        printf("\nQueue Operations:\n");
        printf("1. Insert\n2. Delete\n3. Display\n4. Check if Empty\n5. Check if Full\n6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter the element to insert: ");
                scanf("%d", &element);
                insert(element);
                break;
            case 2:
                delete();
                break;
            case 3:
                display();
                break;
            case 4:
                if (isEmpty()) {
                    printf("Queue is empty.\n");
                } else {
                    printf("Queue is not empty.\n");
                }
                break;
            case 5:
                if (isFull()) {
                    printf("Queue is full.\n");
                } else {
                    printf("Queue is not full.\n");
                }
                break;
            case 6:
                printf("Exiting program.\n");
                exit(0);
            default:
```

```
            printf("Invalid choice! Please try again.\n");
        }
    }
    return 0;
}
```

Output:

```
Queue Operations:
1. Insert
2. Delete
3. Display
4. Check if Empty
5. Check if Full
6. Exit
Enter your choice: 1
Enter the element to insert: 66
Inserted 66 into the queue.

Queue Operations:
1. Insert
2. Delete
3. Display
4. Check if Empty
5. Check if Full
6. Exit
Enter your choice: 2
Deleted 66 from the queue.

Queue Operations:
1. Insert
2. Delete
3. Display
4. Check if Empty
5. Check if Full
6. Exit
Enter your choice: 3
Queue is empty!

Queue Operations:
1. Insert
2. Delete
3. Display
4. Check if Empty
5. Check if Full
6. Exit
Enter your choice: 1
Enter the element to insert: 55
```

```
Queue Operations:
1. Insert
2. Delete
3. Display
4. Check if Empty
5. Check if Full
6. Exit
Enter your choice: 5
Queue is not full.

Queue Operations:
1. Insert
2. Delete
3. Display
4. Check if Empty
5. Check if Full
6. Exit
Enter your choice: 6
Exiting program.

--------------------------------
Process exited after 92.28 seconds with return value 0
Press any key to continue . . .
```

CONCLUSION: n this program, a queue is successfully implemented using an array. The operations — Insert, Delete, Display, isEmpty, and isFull — demonstrate how elements are managed in a FIFO (First In, First Out) manner.